

UNIVERSIDAD CONTINENTAL

Facultad de Ingeniería de Sistemas e Informática



ANÁLISIS Y DISEÑO DE SOFTWARE (NRC:28638)

INFORME DE:

PROYECTO "Sistema de reportes urbanos"

Estudiantes:

Estudiante	Código	PORCENTAJE
MANDUJANO CORONEL GEAN CARLOS	72243697	100%
LAGONES BRICEÑO WILL	75779145	100%
SALAZAR YAURI JAVIER	71648721	70%
QUISPE SALAS MONICA	75712447	100%

Docente:

Rosario Delia Osorio Contreras

Grupo: 01

fecha: 21/04/2026

Huancayo – Perú

2026 – I

ÍNDICE GENERAL

UNIDAD I – FUNDAMENTOS Y MODELADO INICIAL

Capítulo 1. Presentación del Proyecto

- 1.1 ODS Vinculado
- 1.2 Institución Beneficiaria
- 1.3 Problema Identificado
- 1.4 Solución Propuesta

Capítulo 2. Análisis de Necesidades y Requerimientos

- 2.1 Descripción del Problema
- 2.2 Necesidades de los Usuarios
- 2.3 Requerimientos Funcionales (RF)
- 2.4 Requerimientos No Funcionales (RNF)
- 2.5 Requerimientos de Dominio (RD)

Capítulo 3. Modelos Iniciales del Sistema

- 3.1 Diagrama de Contexto
- 3.2 Diagrama de Casos de Uso General
- 3.3 Diagrama de Actividad UML
- 3.4 Modelo Entidad-Relación (E-R)

UNIDAD II – MODELOS DE DISEÑO Y METODOLOGÍA ÁGIL

Capítulo 4. Modelos de Diseño

- 4.1 Diagrama de Clases
- 4.2 Diagrama de Secuencia – Registrar Incidencia
- 4.3 Diagrama de Secuencia – Derivación de Casos
- 4.4 Diagrama de Secuencia – Atención y Seguimiento

Capítulo 5. Metodología de Trabajo (SCRUM)

- 5.1 Justificación de la Metodología
- 5.2 Roles del Equipo Scrum
- 5.3 Product Backlog
- 5.4 Planificación de Sprints
- 5.5 Criterios de Aceptación por Historia de Usuario

Conclusiones y Recomendaciones

Evidencias

Referencias Bibliográficas

UNIDAD I – FUNDAMENTOS Y MODELADO INICIAL

Capítulo 1. Presentación del Proyecto

1.1 ODS Vinculado

El presente proyecto se enmarca dentro del Objetivo de Desarrollo Sostenible N.º 11 de la Agenda 2030 de las Naciones Unidas:

ODS11	Ciudades y Comunidades Sostenibles
	Meta 11.6: Reducir el impacto ambiental negativo per cápita de las ciudades, prestando especial atención a la calidad del aire y la gestión de los desechos municipales y de otro tipo.
	Meta 11.3: Aumentar la urbanización inclusiva y sostenible y la capacidad para la planificación y la gestión participativas, integradas y sostenibles de los asentamientos humanos.

El Sistema de Reportes Urbanos Sostenibles contribuye directamente a este ODS al habilitar un canal tecnológico que conecta a la ciudadanía con la administración municipal, facilitando la identificación, registro y resolución oportuna de incidencias que afectan la habitabilidad del entorno urbano, tales como acumulación de residuos sólidos, fallas en el alumbrado público y situaciones de inseguridad ciudadana.

1.2 Institución Beneficiaria

Las instituciones beneficiarias directas e indirectas del sistema son:

Actor	Tipo de Beneficio	Nivel de Impacto
Municipalidad Provincial de Huancayo	Optimización de recursos operativos y mejora en tiempos de respuesta	Directo – Alto
Municipalidades Distritales (El Tambo, Chilca, Huancayo)	Gestión descentralizada de incidencias por sector	Directo – Alto
Juntas Vecinales y Comités Comunales	Canal oficial de reporte y seguimiento de su zona	Directo – Medio
Ciudadanía en General	Participación activa en la mejora del entorno urbano	Directo – Alto
Ministerio del Ambiente (MINAM)	Datos agregados sobre gestión de residuos urbanos	Indirecto – Medio
OEFA y organismos fiscalizadores	Indicadores de cumplimiento ambiental municipal	Indirecto – Bajo

1.3 Problema Identificado

Actualmente, los ciudadanos de los distritos de Huancayo, El Tambo y Chilca no disponen de un canal formal, digital y trazable para comunicar incidencias urbanas a sus respectivas municipalidades. Los mecanismos existentes presentan las siguientes deficiencias críticas:

- Dependencia de canales informales: las incidencias se reportan por llamadas, visitas o redes sociales, sin registro oficial ni seguimiento.
- Ausencia de trazabilidad: el ciudadano no recibe confirmación ni actualizaciones sobre su reporte.
- Ineficiencia en la asignación de recursos: no se cuenta con información georreferenciada para priorizar intervenciones.
- Falta de métricas de gestión: no es posible generar indicadores para evaluar el desempeño institucional.
- Duplicación de esfuerzos: pueden asignarse recursos al mismo caso mientras otras incidencias quedan sin atender.

Esta problemática genera una degradación progresiva de la infraestructura urbana, afecta la calidad de vida de la población y erosiona la confianza ciudadana en la gestión municipal. Según datos del Índice de Satisfacción Ciudadana Municipal (INDECI, 2023), más del 62% de los ciudadanos de ciudades intermedias del Perú considera que sus reportes de servicios públicos no reciben atención oportuna.

1.4 Solución Propuesta – Plataforma PIGIU

Se propone el desarrollo de la Plataforma Integral de Gestión de Incidencias Urbanas (PIGIU). En dos capas.

1.4.1 Aplicación Móvil Ciudadana (Android/iOS)

Canal principal de ingreso de incidencias al sistema. Sus funcionalidades clave son:

- Registro de incidencias con captura fotográfica automática desde la cámara del dispositivo móvil.
- Geolocalización automática por GPS con mapa interactivo para confirmación o ajuste manual de la ubicación.
- Clasificación guiada de la incidencia en categorías: Residuos Sólidos, Alumbrado Público, Seguridad Ciudadana, Vías y Pavimento, Áreas Verdes y Otros.
- Generación automática de número de ticket con código QR para seguimiento offline.
- Notificaciones push en tiempo real: confirmación de registro, cambio de estado y resolución.
- Historial de reportes del usuario con línea de tiempo de atención.
- Módulo de valoración del servicio: el ciudadano puede calificar la atención recibida (1-5 estrellas).

1.4.2 Panel Administrativo Web (Municipalidad)

Interfaz de gestión para funcionarios municipales y técnicos operativos:

- Mapa de calor interactivo con incidencias activas clasificadas por tipo, sector y nivel de urgencia.
- Módulo de derivación automática o manual de casos a la unidad orgánica responsable (Gerencia de Servicios Públicos, Seguridad Ciudadana, Obras Públicas).
- Gestión del ciclo de vida completo del reporte: Pendiente → En Revisión → Asignado → En Atención → Resuelto → Cerrado.
- Módulo de análisis y reportes: estadísticas mensuales, tiempos promedio de atención, sectores con mayor incidencia, KPIs de desempeño por unidad.
- Integración con el Catastro Municipal para validar jurisdicción e identificar infraestructura afectada.
- Módulo de comunicación bidireccional: el funcionario puede solicitar información adicional al ciudadano a través de la app.

1.4.3 Arquitectura Tecnológica

Componente	Tecnología Propuesta	Justificación
Backend / API REST	Node.js + Express.js	Alto rendimiento para I/O concurrente, amplia comunidad
Base de Datos Relacional	PostgreSQL + PostGIS	Soporte nativo para datos geoespaciales
Almacenamiento de Imágenes	Amazon S3 / Cloudinary	Escalabilidad y gestión optimizada de archivos multimedia
Aplicación Móvil	React Native	Desarrollo multiplataforma (iOS/Android) desde una sola base de código
Panel Administrativo Web	React.js + Material UI	Interfaz moderna, responsiva y de rápida curva de aprendizaje
Notificaciones Push	Firebase Cloud Messaging (FCM)	Servicio confiable y gratuito de Google
Mapas e Integración GIS	OpenStreetMap + Leaflet.js	Open source, sin costo de licencia, personalizable
Autenticación	JWT + OAuth 2.0	Estándar de la industria, compatible con Google/Facebook
Despliegue	Docker + AWS EC2 / Heroku	Contenedores para portabilidad y escalabilidad

Capítulo 2. Análisis de Necesidades y Requerimientos

2.1 Descripción del Problema

El proceso actual de gestión de incidencias urbanas en el ámbito municipal de Huancayo puede describirse como un flujo no estructurado, reactivo y sin trazabilidad. El siguiente análisis causal sintetiza la problemática:

Causa Raíz	Efecto Directo	Impacto en la Ciudadanía
Ausencia de sistema digital de reportes	Reportes no registrados o perdidos	Incidencias sin atención durante semanas o meses
Sin georreferenciación de incidencias	Asignación ineficiente de cuadrillas	Mayor tiempo de respuesta y mayor costo operativo
Sin notificaciones al ciudadano	Desconfianza y percepción de abandono	Desmotivación para futuros reportes
Sin métricas de desempeño municipal	Imposible rendir cuentas	Opacidad en la gestión pública
Canales informales y dispersos	Duplicación o pérdida de casos	Inequidad en la atención según sector geográfico

2.2 Necesidades de los Usuarios

2.2.1 Ciudadano Reportante

- Necesidad N1: Registrar incidencias de forma rápida (menos de 60 segundos) desde su dispositivo móvil sin necesidad de formación técnica.
- Necesidad N2: Recibir confirmación inmediata de que su reporte fue recibido y un número de seguimiento único.
- Necesidad N3: Conocer en todo momento el estado actualizado de su reporte y el tiempo estimado de atención.
- Necesidad N4: Poder adjuntar evidencia fotográfica que respalde la veracidad de la incidencia.
- Necesidad N5: Tener la posibilidad de calificar la atención recibida para contribuir a la mejora del servicio.

2.2.2 Técnico Operativo Municipal

- Necesidad N6: Visualizar en un mapa interactivo todas las incidencias activas de su sector, priorizadas por urgencia.
- Necesidad N7: Recibir asignación directa de los casos que le corresponden atender, con detalle de ubicación y evidencia.
- Necesidad N8: Actualizar el estado del reporte desde el campo mediante la app o el panel web.

2.2.3 Funcionario Municipal

- Necesidad N9: Derivar casos automáticamente o manualmente a las unidades orgánicas responsables.
- Necesidad N10: Generar reportes estadísticos mensuales por tipo de incidencia, sector y tiempo de atención.
- Necesidad N11: Identificar 'puntos calientes' urbanos para planificación preventiva y presupuestal.

2.2.4 Alcaldía

- Necesidad N12: Disponer de un dashboard ejecutivo con KPIs de desempeño en tiempo real.
- Necesidad N13: Generar informes de transparencia para publicación en el portal de la municipalidad, cumpliendo con la Ley 27806.

2.3 Requerimientos Funcionales (RF)

Los requerimientos funcionales se organizan en cuatro procesos clave del sistema: Registro, Derivación de Casos, Atención de Casos y Seguimiento.

RF-01	Registro de Incidencia con Evidencia Fotográfica y GPS
Descripción	El sistema debe permitir al ciudadano autenticado registrar una nueva incidencia urbana capturando una o varias fotografías desde la cámara del dispositivo y obteniendo la ubicación geográfica exacta de manera automática mediante GPS. El usuario podrá ajustar manualmente el punto en el mapa si la señal GPS no es precisa.
Actor	Ciudadano Reportante
Precondición	El usuario debe estar autenticado en el sistema y tener habilitados los permisos de cámara y ubicación en su dispositivo.
Flujo Principal	1. El ciudadano abre la app y selecciona 'Nuevo Reporte'. 2. El sistema activa la cámara y el módulo GPS. 3. El ciudadano captura la fotografía y confirma la ubicación. 4. El ciudadano ingresa descripción textual. 5. El sistema valida los datos y genera el ticket.
Criterios de Aceptación	CA-01.1: El sistema genera un número de ticket único en formato TK-[AÑO]-[SECUENCIAL] en menos de 3 segundos tras el envío. CA-01.2: La fotografía adjunta debe tener mínimo 640x480px; el sistema rechaza imágenes de menor resolución con mensaje claro. CA-01.3: La ubicación GPS debe tener precisión mínima de 50 metros; si no se alcanza, el sistema alerta al usuario. CA-01.4: El sistema envía confirmación por notificación push y correo electrónico dentro de los 60 segundos siguientes al registro. CA-01.5: El reporte queda almacenado en estado 'Pendiente' y es visible en el panel administrativo en tiempo real.

RF-02	Clasificación de Incidencia por Categoría y Subcategoría
Descripción	El sistema debe ofrecer una taxonomía de dos niveles: Categoría principal (Alumbrado Público, Residuos Sólidos, Seguridad Ciudadana, Vías y Pavimento, Áreas Verdes, Infraestructura Pública) y Subcategoría específica, permitiendo una derivación precisa al área responsable.

Criterios de Aceptación	CA-02.1: La taxonomía de categorías es configurable por el administrador del sistema sin requerir despliegue de código.CA-02.2: El sistema sugiere automáticamente la categoría más probable basándose en palabras clave de la descripción ingresada (IA básica).CA-02.3: La categoría seleccionada determina automáticamente la unidad orgánica a la que se derivará el reporte.CA-02.4: Cada categoría tiene un SLA (tiempo máximo de atención) predefinido configurable: Urgente=24h, Normal=72h, Baja=168h.
--------------------------------	---

RF-03	Autenticación y Registro de Usuarios
Descripción	El sistema debe permitir el registro e inicio de sesión de ciudadanos mediante correo electrónico/contraseña, cuenta Google o cuenta Facebook. El acceso al panel administrativo está restringido a funcionarios municipales con credenciales institucionales.
Criterios de Aceptación	CA-03.1: El proceso de registro ciudadano no debe requerir más de 5 campos obligatorios: nombre completo, DNI, correo, contraseña, distrito.CA-03.2: La contraseña debe cumplir política de seguridad: mínimo 8 caracteres, una mayúscula, un número.CA-03.3: El sistema envía correo de verificación; la cuenta se activa solo tras confirmación.CA-03.4: Los funcionarios municipales se autentican con credenciales LDAP/AD de la municipalidad.CA-03.5: El sistema implementa bloqueo de cuenta tras 5 intentos fallidos consecutivos.

PROCESO 2: DERIVACIÓN DE CASOS

RF-04	Derivación Automática de Incidencias a Unidad Responsable
Descripción	El sistema debe derivar automáticamente cada incidencia registrada a la unidad orgánica municipal correspondiente según la categoría seleccionada. Si la derivación automática no es posible, el supervisor puede reasignar manualmente.
Reglas de Derivación	- Residuos Sólidos → Subgerencia de Limpieza Pública - Alumbrado Público → Subgerencia de Infraestructura y Obras - Seguridad Ciudadana → Gerencia de Seguridad Ciudadana / SERENAZGO - Vías y Pavimento → Subgerencia de Obras y Mantenimiento Vial - Áreas Verdes → Subgerencia de Parques y Jardines
Criterios de Aceptación	CA-04.1: La derivación automática ocurre dentro de los 5 minutos posteriores al registro del reporte.CA-04.2: El responsable de la unidad recibe notificación push, correo y alerta en el panel al recibir un nuevo caso.CA-04.3: El sistema registra en bitácora el usuario, fecha/hora y motivo de toda reasignación manual.CA-04.4: Si un caso no es derivado dentro de 60 minutos, el sistema escala automáticamente al supervisor de guardia.CA-04.5: El ciudadano recibe notificación cuando su caso es asignado a una unidad, indicando el nombre de la unidad responsable.

RF-05	Panel de Mapa Interactivo para Gestión de Incidencias
Descripción	El panel administrativo debe mostrar un mapa interactivo con todas las incidencias activas representadas con marcadores diferenciados por tipo y estado, incluyendo capacidad de filtrado, clustering de puntos cercanos y mapa de calor por densidad.
Criterios de Aceptación	CA-05.1: El mapa carga con más de 1,000 puntos activos en menos de 3 segundos.CA-05.2: El usuario puede filtrar por categoría, estado, rango de fechas y sector geográfico.CA-05.3: Al hacer clic en un marcador, se despliega un popup con: foto, descripción, fecha, estado actual y datos del ciudadano (anonimizados para niveles bajos de acceso).CA-05.4: El mapa se actualiza en tiempo real (WebSockets) al ingresar nuevas incidencias sin necesidad de recargar la página.

PROCESO 3: ATENCIÓN DE CASOS

RF-06	Gestión del Ciclo de Vida del Reporte
Descripción	El sistema debe gestionar el ciclo de vida completo del reporte a través de los estados: Pendiente → En Revisión → Asignado → En Atención → Resuelto → Cerrado. Cada transición de estado debe registrar: usuario responsable, fecha/hora y comentario opcional.
Criterios de Aceptación	CA-06.1: Solo usuarios con rol autorizado pueden transicionar a estados específicos (ej: solo el técnico asignado puede marcar 'En Atención').CA-06.2: Al marcar 'Resuelto', el técnico debe adjuntar fotografía del estado final como evidencia obligatoria.CA-06.3: El sistema calcula automáticamente el tiempo total de atención (desde Pendiente hasta Resuelto) y lo muestra en el reporte.CA-06.4: Un reporte no puede pasar de 'Pendiente' a 'Resuelto' directamente; debe cumplir el flujo establecido.CA-06.5: Los reportes en estado 'Asignado' sin actualización por más de 24h generan una alerta automática al supervisor.

RF-07	Generación de Reportes Estadísticos Administrativos
Descripción	El panel debe generar reportes estadísticos periódicos (mensual, trimestral, anual) con KPIs de gestión exportables en formato PDF y Excel.
KPIs Requeridos	• Total de reportes por período / Tasa de resolución (%) • Tiempo promedio de primera respuesta (TPPR) • Tiempo promedio de resolución (TPR) por categoría • Ranking de sectores con mayor incidencia • Tasa de satisfacción ciudadana (promedio de calificaciones) • Carga de trabajo por unidad orgánica
Criterios de Aceptación	CA-07.1: Los reportes se generan en menos de 10 segundos para períodos de hasta 12 meses.CA-07.2: Los datos exportados en Excel incluyen filas individuales por reporte con todos los campos del ciclo de vida.CA-07.3: El informe PDF incluye gráficos de barras, línea y pastel generados automáticamente.CA-07.4: El sistema permite programar el envío automático por correo del reporte mensual a los funcionarios configurados.

RF-08	Notificaciones Push y Seguimiento del Reporte
Descripción	El sistema debe notificar al ciudadano mediante notificación push y/o correo electrónico cada vez que el estado de su reporte cambie, detallando el nuevo estado, la unidad responsable y el tiempo estimado de resolución.
Criterios de Aceptación	CA-08.1: Las notificaciones push se entregan en menos de 30 segundos tras el cambio de estado.CA-08.2: El ciudadano puede configurar sus preferencias de notificación (push, correo o ambas).CA-08.3: Al estado 'Resuelto', la notificación incluye la foto del trabajo terminado adjuntada por el técnico.CA-08.4: El ciudadano puede acceder a su historial completo de reportes y el detalle de cada interacción desde la app.

RF-09	Valoración del Servicio y Cierre del Reporte
Descripción	Una vez marcado el reporte como 'Resuelto', el ciudadano recibe una solicitud de valoración del servicio (escala 1-5 estrellas) y un campo de comentario opcional. El reporte pasa a estado 'Cerrado' tras la valoración o automáticamente a las 72 horas si no hay respuesta.
Criterios de Aceptación	CA-09.1: La solicitud de valoración se envía automáticamente 2 horas después de marcarse como 'Resuelto'.CA-09.2: Si el ciudadano califica con 1 o 2 estrellas, el sistema genera una alerta al supervisor para revisión.CA-09.3: Las valoraciones se consolidan en el KPI de satisfacción del dashboard ejecutivo.CA-09.4: El ciudadano puede reabrir un reporte cerrado dentro de las 48 horas si considera que no fue resuelto correctamente.

2.4 Requerimientos No Funcionales (RNF)

ID	Categoría	Descripción	Métrica de Cumplimiento
RNF-01	Disponibilidad	El sistema debe estar operativo el 99.5% del tiempo mensual, con mantenimiento programado fuera de horario laboral.	Tiempo de inactividad no planificado ≤ 3.6 horas/mes
RNF-02	Rendimiento	El panel administrativo debe cargar el mapa de incidencias con más de 1,000 puntos en menos de 3 segundos.	Medición con Lighthouse: LCP < 3s
RNF-03	Usabilidad	El proceso de registro de una incidencia en la app móvil no debe requerir más de 4 pasos ni tomar más de 60 segundos.	Prueba de usabilidad con 10 usuarios no técnicos
RNF-04	Seguridad	Los datos personales de los ciudadanos deben almacenarse cifrados (AES-256) y las comunicaciones con TLS 1.3.	Auditoría de seguridad con OWASP ZAP
RNF-05	Privacidad	El sistema debe cumplir la Ley N.º 29733 (Ley de Protección de Datos)	Certificación de conformidad legal

		Personales del Perú) y su reglamento.	
RNF-06	Escalabilidad	La arquitectura debe soportar hasta 10,000 usuarios concurrentes sin degradación del servicio.	Prueba de carga con Apache JMeter
RNF-07	Portabilidad	La app móvil debe funcionar en Android 8.0+ e iOS 13.0+, cubriendo el 95% de los dispositivos del mercado peruano.	Pruebas en 10 dispositivos físicos representativos
RNF-08	Mantenibilidad	El código fuente debe seguir estándares de documentación y tener cobertura de pruebas unitarias $\geq 70\%$.	Reporte de cobertura con Jest/Coverage

2.5 Requerimientos de Dominio (RD)

RD-01	Cumplimiento de Normativa de Transparencia Gubernamental
Normativa Aplicable	Ley N.° 27806 – Ley de Transparencia y Acceso a la Información Pública del Perú; y los lineamientos del Decreto Supremo N.° 072-2003-PCM que establece el Reglamento de dicha ley.
Obligación del Sistema	El sistema debe generar y publicar automáticamente en el portal web municipal un resumen estadístico mensual de incidencias: número de reportes recibidos, atendidos, pendientes y tiempos de atención por sector. Esta publicación debe realizarse antes del día 5 del mes siguiente.
Criterios de Aceptación	CA-RD01.1: El informe mensual es generado automáticamente el día 1 de cada mes y está disponible en el portal público en formato PDF y HTML. CA-RD01.2: El informe público muestra datos agregados, sin información personal identificable de los ciudadanos. CA-RD01.3: Los datos son auditables por el Órgano de Control Institucional (OCI) mediante exportación de logs.

RD-02	Integración con el Plan PIGARS de Gestión de Residuos Sólidos
Normativa Aplicable	Ley N.° 27314 – Ley General de Residuos Sólidos y su modificatoria el D.Leg. N.° 1278 (Ley de Gestión Integral de Residuos Sólidos). El PIGARS (Plan Integral de Gestión Ambiental de Residuos Sólidos) es el instrumento de planificación municipal de gestión de residuos según el D.S. N.° 014-2017-MINAM.
Obligación del Sistema	Los reportes de la categoría 'Residuos Sólidos' deben integrarse con los parámetros del PIGARS municipal: horarios y rutas de recolección vigentes, zonas de disposición temporal autorizadas y calendario de recojo diferenciado (orgánicos/inorgánicos). El sistema debe validar que la incidencia no corresponde a un punto de acopio autorizado antes de generar la alerta.
Criterios de Aceptación	CA-RD02.1: El sistema tiene cargadas las rutas PIGARS y el horario de recolección actualizable por el administrador. CA-RD02.2: Si una incidencia de basura se reporta en zona con recojo programado en las próximas 4 horas, el sistema sugiere al

	ciudadano esperar el servicio antes de confirmar el reporte.CA-RD02.3: Las rutas PIGARS son visibles en el mapa del panel administrativo como capa independiente.
--	---

RD-03	Integración con el Catastro Municipal para Validación de Jurisdicción
Normativa Aplicable	D.S. N.° 002-89-JUS y Ley N.° 28294 (Sistema Nacional Integrado de Catastro). La Ley Orgánica de Municipalidades (Ley N.° 27972) establece que la municipalidad solo es competente para atender incidencias dentro de su ámbito jurisdiccional.
Obligación del Sistema	El sistema debe integrar la capa GIS del Catastro Municipal (límites distritales, zonificación y nomenclatura vial) para: (a) Validar automáticamente que la ubicación GPS de la incidencia se encuentra dentro del ámbito de competencia de la municipalidad receptora; (b) Identificar la vía pública o infraestructura afectada con su código catastral.
Criterios de Aceptación	CA-RD03.1: Si la incidencia se ubica fuera del límite jurisdiccional, el sistema lo notifica al ciudadano y le sugiere el municipio competente, sin rechazar el reporte.CA-RD03.2: El sistema resuelve la dirección postal a partir de coordenadas GPS (geocodificación inversa) y la adjunta automáticamente al reporte.CA-RD03.3: Los datos del catastro se actualizan al menos semestralmente desde la fuente oficial.

RD-04	Protección de Datos Personales (NUEVO)
Normativa Aplicable	Ley N.° 29733 – Ley de Protección de Datos Personales del Perú y D.S. N.° 003-2013-JUS (Reglamento). El sistema procesa datos sensibles de ciudadanos (DNI, correo, ubicación en tiempo real), lo que lo clasifica como banco de datos de administración pública sujeto a registro en la ANPD.
Obligación del Sistema	El sistema debe registrar el banco de datos ante la Autoridad Nacional de Protección de Datos Personales (ANPD), obtener consentimiento explícito del ciudadano al momento del registro, y garantizar los derechos ARCO (Acceso, Rectificación, Cancelación y Oposición).
Criterios de Aceptación	CA-RD04.1: El proceso de registro incluye checkbox obligatorio de aceptación de política de privacidad, con enlace al documento completo.CA-RD04.2: El ciudadano puede solicitar desde la app la eliminación de su cuenta y datos asociados; el sistema procesa la solicitud en ≤ 15 días hábiles.CA-RD04.3: El acceso a datos personales de ciudadanos en el panel administrativo está restringido según rol: los niveles operativos solo ven el nombre y ticket, los datos completos solo para fiscalía o jefatura.

Capítulo 3. Modelos Iniciales del Sistema

3.1 Diagrama de Contexto

El Diagrama de Contexto representa el sistema PIGIU como una caja negra, mostrando las entidades externas que interactúan con él y los flujos de información que intercambian:

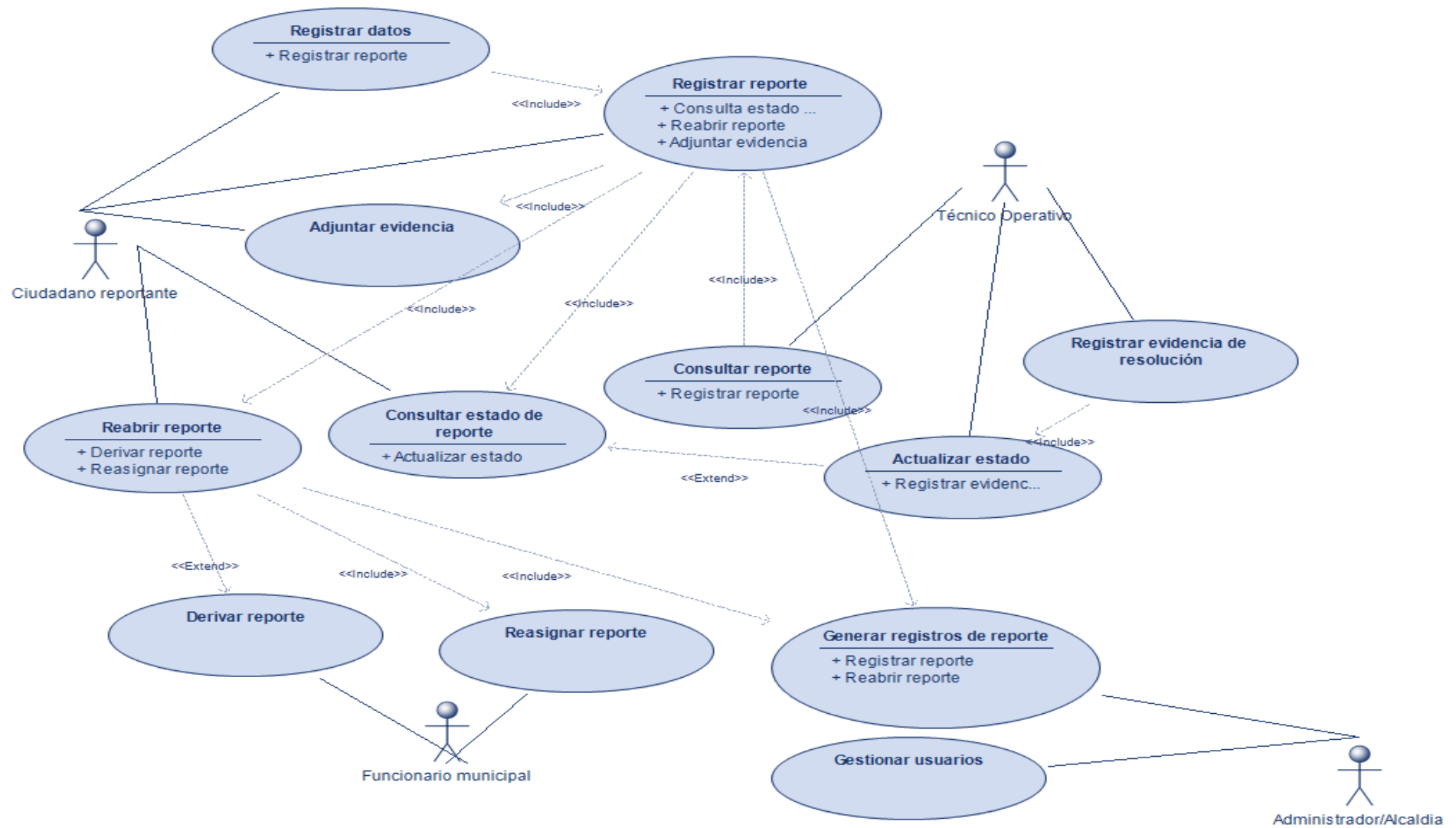


Nota: El símbolo → indica flujo de entrada al sistema; ← indica flujo de salida. Las entidades externas actúan como proveedores y consumidores de información del sistema PIGIU.

3.2 Diagrama de Casos de Uso – Vista General

El diagrama de casos de uso del sistema PIGIU representa la interacción entre los principales actores involucrados en la gestión de incidencias urbanas: el Ciudadano Reportante, el Técnico Operativo, el Funcionario Municipal y el Administrador o AEl. El Ciudadano Reportante inicia el proceso al registrar una incidencia, ingresando los datos correspondientes y adjuntando evidencia fotográfica. Asimismo, puede consultar el estado de su reporte en cualquier momento y, en caso de no estar conforme con la atención recibida, solicitar la reapertura del caso. El Técnico Operativo participa en la fase de atención, consultando los reportes asignados, actualizando su estado y registrando la evidencia de resolución una vez culminada la intervención. Por su parte, el Funcionario Municipal desempeña un papel clave en la supervisión y control del proceso, ya que puede derivar reportes a las áreas competentes, reasignarlos cuando sea necesario y gestionar los casos reabiertos para asegurar una atención adecuada. Finalmente, el Administrador o Alcaldía tiene acceso a funciones estratégicas, como la generación de registros y reportes consolidados, así como la gestión de usuarios del sistema. En conjunto, este modelo refleja una solución integral, colaborativa y orientada a optimizar la atención, seguimiento y control de las incidencias urbanas.

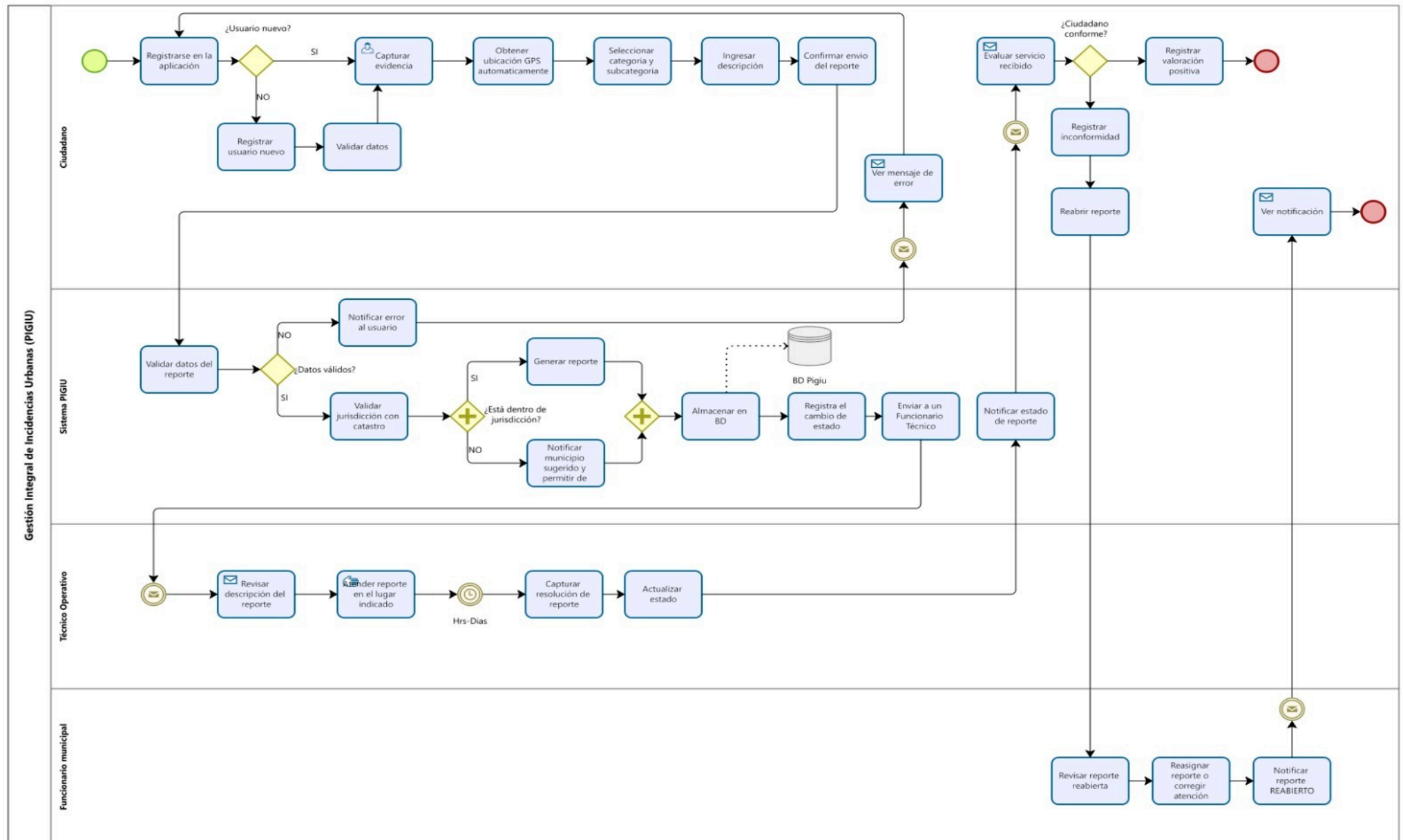
lcaldía.



Actor	Casos de Uso Asociados
Ciudadano Reportante	CU-01 Registrar datos; CU-02 Registrar reportes; CU-03 Adjuntar evidencia; CU-04 Consultar estado de reporte; CU-05 Reabrir reporte.
Técnico operativo	CU-06 Consultar reporte CU-07 Actualizar Estado de reporte; CU-08 Adjuntar Evidencia de Resolución.
Funcionario municipal	CU-09 Derivar reporte, CU-10 Reasignar reporte.
Administrador / Alcaldía	CU-11 Generar Reportes Estadísticos; CU-12 Gestionar usuarios.

3.3 Diagrama de Actividad UML – Proceso Completo de Gestión

El modelo BPMN desarrollado en Bizagi para la Plataforma Integral de Gestión de Incidencias Urbanas, articula la interacción coordinada de cuatro actores principales: el Ciudadano, el Sistema PIGIU, el Técnico Operativo y el Funcionario Municipal. El proceso inicia cuando el ciudadano registra una incidencia mediante la aplicación, proporcionando evidencia fotográfica, ubicación geográfica, categoría y descripción del problema. Posteriormente, el Sistema PIGIU valida la información ingresada, verifica la jurisdicción territorial mediante integración con el catastro y genera automáticamente el reporte, almacenando en la base de datos y asignándole al técnico correspondiente. El Técnico Operativo recibe la notificación, revisa el caso, se desplaza al lugar de la incidencia, ejecuta la atención y registra la evidencia de resolución, actualizando el estado del reporte. Una vez resuelto, el sistema notifica al ciudadano, quien evalúa la calidad del servicio recibido. Si el ciudadano manifiesta conformidad, el caso se cierra satisfactoriamente. En caso contrario, se registra la inconformidad y el reporte es reabierto. En esta etapa, el Funcionario Municipal asume un rol fundamental, ya que revisa el caso reabierto, evalúa la atención brindada y procede a asignar el reporte o disponer acciones correctivas, garantizando así la adecuada resolución de la incidencia y la mejora continua del servicio municipal.



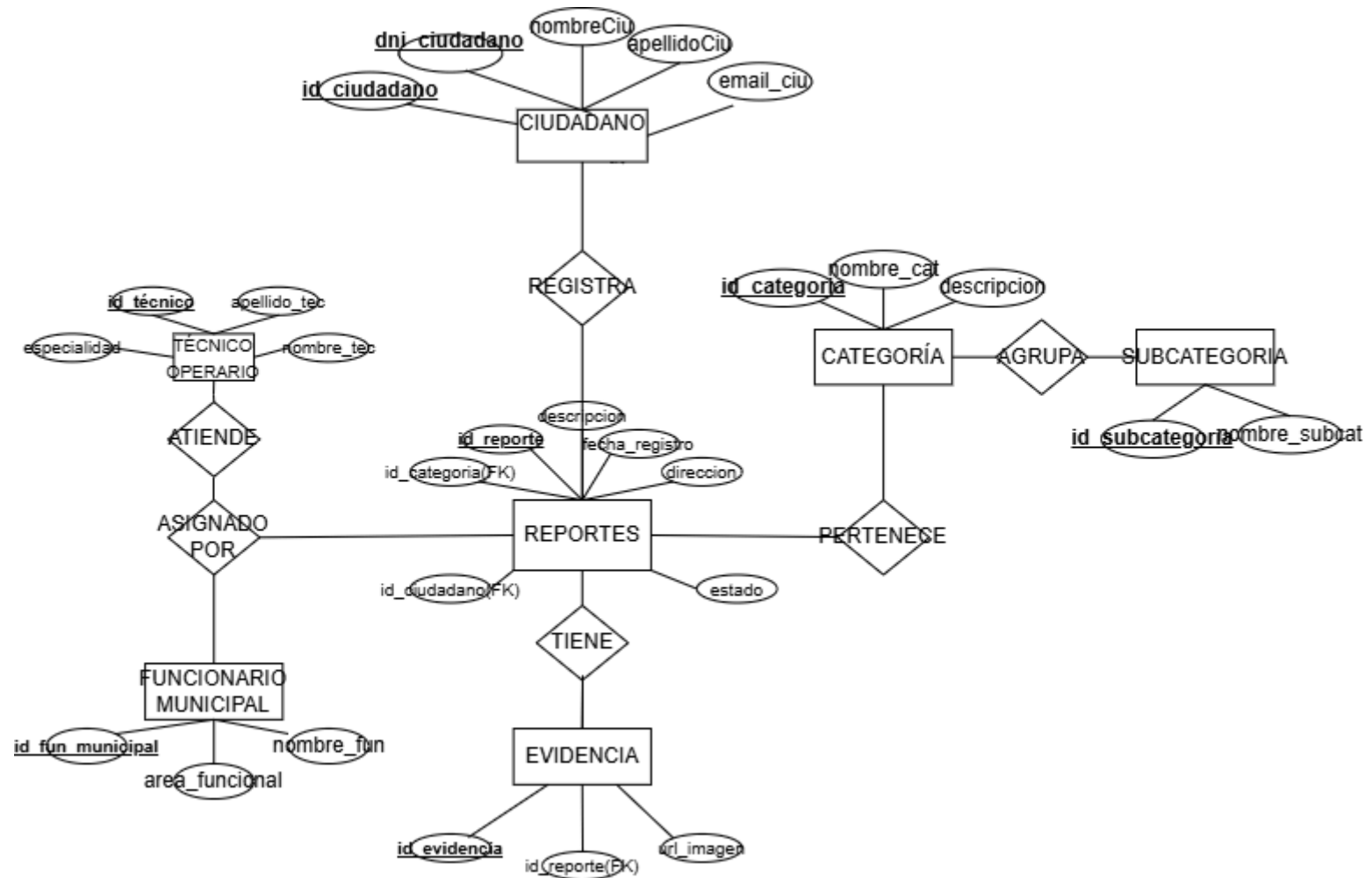
3.4 Modelo Entidad-Relación (E-R)

El modelo entidad-relación del Sistema de Gestión Integral de Incidencias Urbanas, está conformado por varias entidades principales que permiten administrar de manera estructurada el registro, seguimiento y atención de incidencias urbanas. La entidad ciudadano almacena la información de los usuarios que reportan incidencias, incluyendo atributos como identificador, DNI, nombres, apellidos y correo electrónico. Un ciudadano puede registrar uno o varios reportes, estableciéndose así una relación de uno a muchos con la entidad reporte. Esta última constituye la entidad central del sistema y contiene información esencial como el identificador del reporte, descripción de la incidencia, fecha de registro, dirección, estado, así como las claves foráneas que lo vinculan con el ciudadano y la categoría correspondiente.

La entidad categoría clasifica los diferentes tipos de incidencias mediante atributos como identificador, nombre y descripción. Cada categoría puede agrupar múltiples subcategorías, generando una relación de uno a muchos con la entidad subcategoría, la cual permite una clasificación más específica de los reportes. Asimismo, cada reporte pertenece a una única categoría, mientras que una categoría puede estar asociada a múltiples reportes.

Para respaldar la información reportada, la entidad evidencia almacena archivos o imágenes relacionados con cada incidencia. Esta entidad incluye atributos como identificador de evidencia, URL de la imagen y el identificador del reporte al que pertenece. La relación entre reportes y evidencia es de uno a muchos, ya que un reporte puede contar con varias evidencias, pero cada evidencia está asociada a un único reporte.

En el proceso de atención intervienen las entidades técnico operativo y funcionario municipal. El técnico operativo, identificado por atributos como código, nombres, apellidos y especialidad, es responsable de atender las incidencias asignadas. Por su parte, el funcionario municipal participa en la asignación y supervisión de los reportes, contando con atributos como identificador, nombre y área funcional. La relación entre estas entidades y reportes refleja el flujo operativo del sistema: el funcionario municipal asigna los reportes y el técnico operativo los atiende, garantizando una gestión eficiente desde el registro hasta la resolución de cada incidencia.



UNIDAD II – MODELOS DE DISEÑO Y METODOLOGÍA ÁGIL

Capítulo 4. Modelos de Diseño

4.1 Diagrama de Clases del Sistema

El diagrama de clases del Sistema de Gestión Integral de Incidencias Urbanas, muestra la estructura del sistema y cómo interactúan sus principales componentes. La clase central es reporte, ya que almacena la información esencial de cada incidencia, como descripción, ubicación, fecha, hora y estado. Además, permite confirmar el registro, actualizar su estado, derivarlo a otras áreas y reasignarlo cuando sea necesario.

La clase ciudadano reportante representa a la persona que registra la incidencia. Contiene sus datos personales y le permite realizar acciones como registrar una incidencia, consultar el estado de sus reportes, valorar la atención recibida o reabrir un caso cerrado. Cada ciudadano puede generar varios reportes a lo largo del tiempo.

Para complementar la información, la clase evidencia almacena imágenes, archivos u otros documentos que respalden el reporte. Asimismo, las clases categoría principal y subcategoría organizan y clasifican las incidencias, permitiendo una identificación más precisa del problema reportado.

En la gestión operativa intervienen el funcionario municipal, quien supervisa, deriva y reasigna los reportes, y el técnico operativo, encargado de atender las incidencias en campo, actualizar su estado y registrar la resolución. Por otro lado, la clase consultar estado permite hacer seguimiento al avance del reporte y registrar observaciones durante su atención.

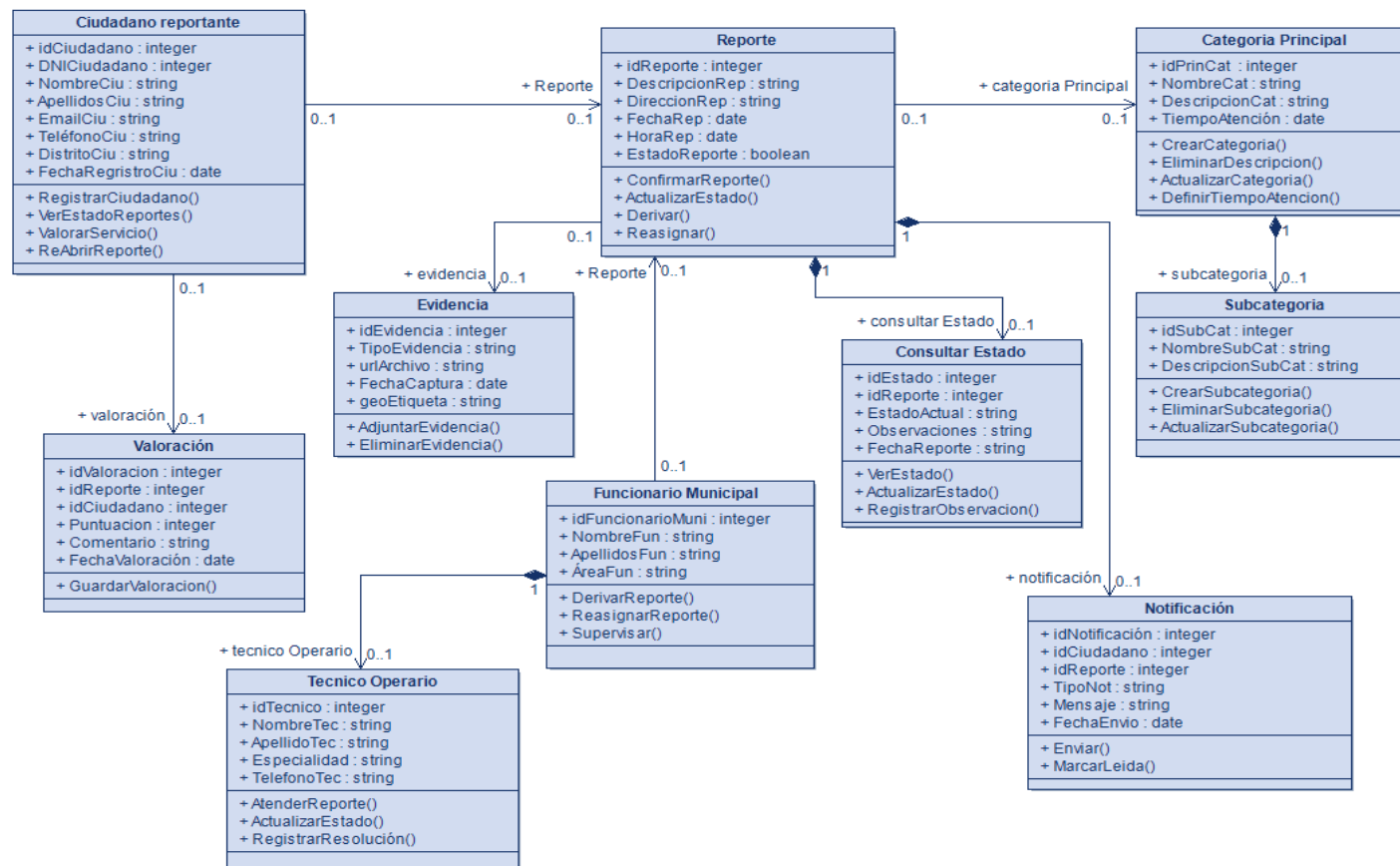
Finalmente, las clases de notificación y valoración fortalecen la interacción con el ciudadano. La primera facilita el envío de avisos sobre cambios en el estado del reporte, mientras que la segunda permite evaluar la calidad del servicio mediante puntuaciones y comentarios. En conjunto, estas clases garantizan una gestión integral, eficiente y orientada al usuario.

UNIDAD II – MODELOS DE DISEÑO Y METODOLOGÍA ÁGIL

Capítulo 4. Modelos de Diseño

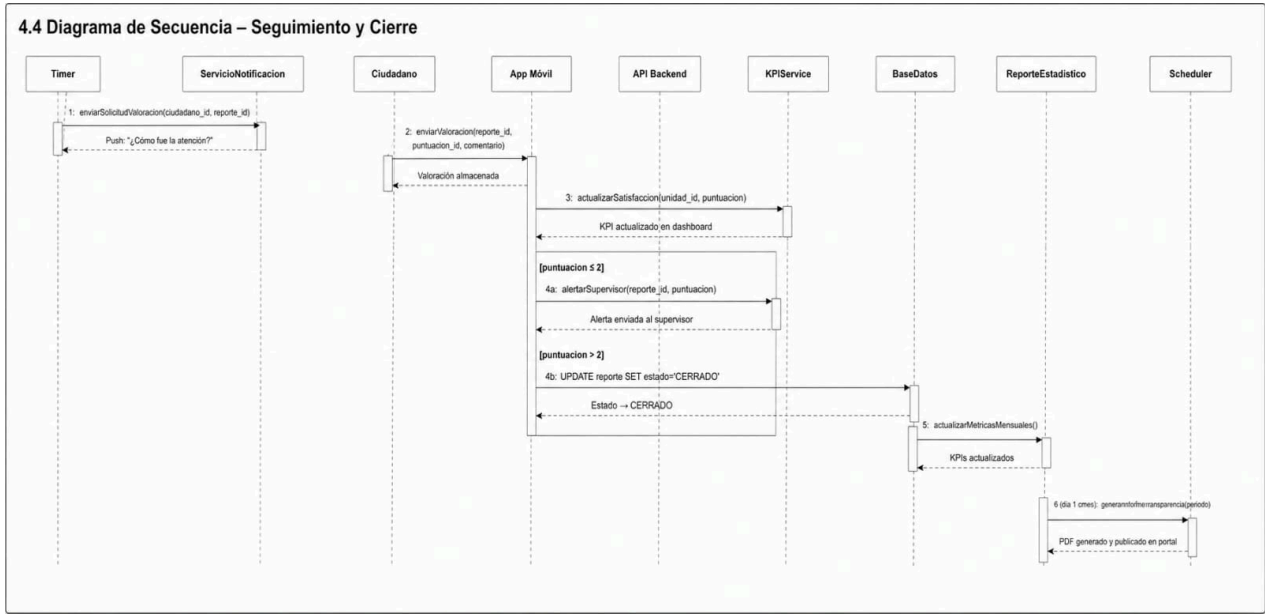
4.1 Diagrama de Clases del Sistema

El diagrama de clases presenta la estructura orientada a objetos del sistema PIGIU, con sus atributos, métodos y relaciones de herencia, composición y asociación:



4.2 Diagrama de Secuencia – RF-01: Registrar Incidencia

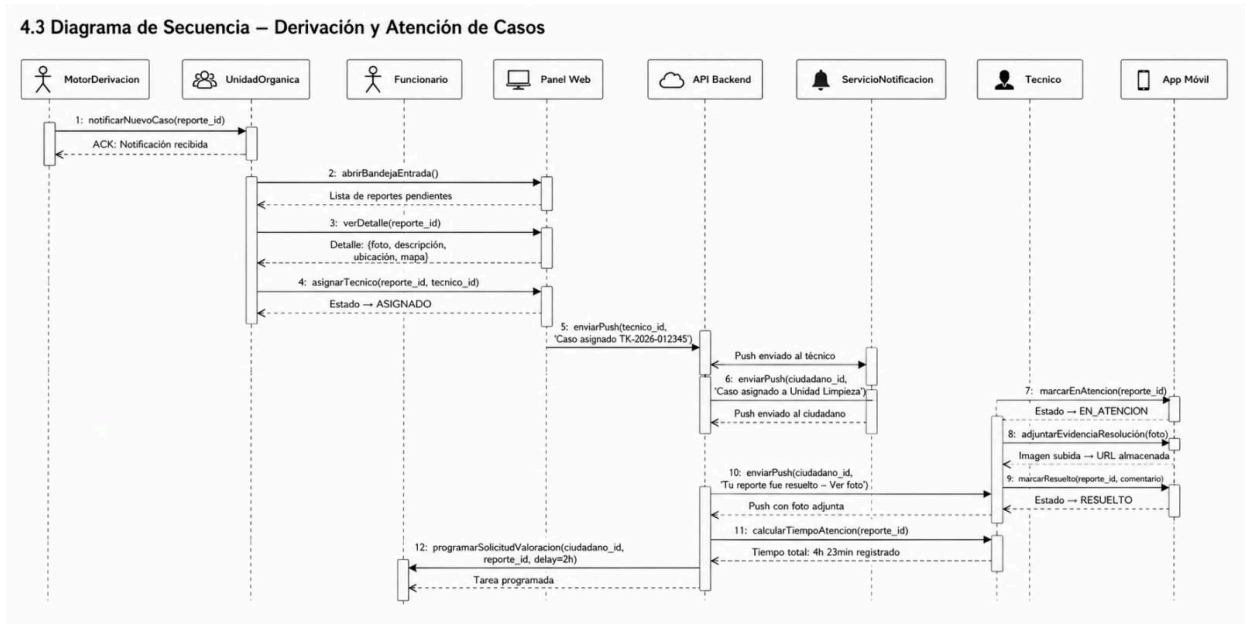
El diagrama muestra la interacción temporal entre los objetos del sistema durante el registro de una incidencia:



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	Ciudadano → App Móvil	seleccionarNuevoReporte()	Pantalla de captura activa
2	App → ServicioGPS	obtenerUbicacion()	Coordenadas {lat, lon, precision}
3	App → Cámara	capturarFoto()	Imagen en buffer
4	App → API Backend	POST /reportes {foto, lat, lon, descripcion, categoria_id}	202 Accepted
5	API → ServicioGIS	validarJurisdiccion(lat, lon)	Distrito: {nombre, municipalidad_id} Error: FueraJurisdiccion
6	API → ServicioAlmacenamiento	subirImagen(imagen, reporte_id)	URL pública de la imagen
7	API → BaseDatos	INSERT INTO reporte (...) VALUES (...)	id_reporte: 12345
8	API → ReporteService	generarTicket(id_reporte)	TK-2026-012345
9	API → MotorDerivacion	derivarAutomaticamente(reporte_id, categoria_id)	unidad_id: 3 (Limpieza Pública)

10	API → ServicioNotificacion	enviarPush(ciudadano_id, 'Reporte registrado: TK-2026-012345')	ACK: Push enviado
11	API → PanelAdmin (WebSocket)	broadcast(nuevoReporte)	Mapa actualizado en tiempo real
12	API → App Móvil	201 Created {ticket: 'TK-2026-012345', estado: 'PENDIENTE'}	Pantalla de confirmación

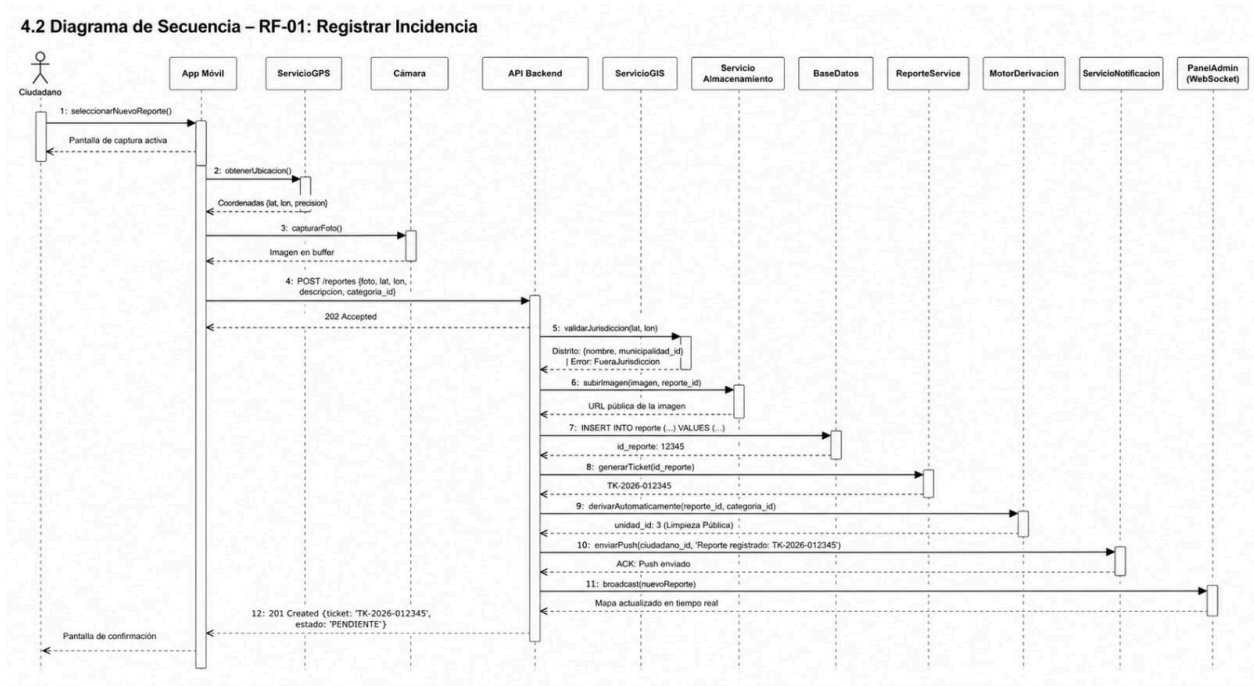
4.3 Diagrama de Secuencia – Derivación y Atención de Casos



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	MotorDerivacion → UnidadOrganica	notificarNuevoCaso(reporte_id)	ACK: Notificación recibida
2	Funcionario → Panel Web	abrirBandejaEntrada()	Lista de reportes pendientes
3	Funcionario → Panel Web	verDetalle(reporte_id)	Detalle: {foto, descripción, ubicación, mapa}
4	Funcionario → Panel Web	asignarTecnico(reporte_id, tecnico_id)	Estado → ASIGNADO
5	API → ServicioNotificacion	enviarPush(tecnico_id, 'Caso asignado TK-2026-012345')	Push enviado al técnico
6	API → ServicioNotificacion	enviarPush(ciudadano_id, 'Caso asignado a Unidad Limpieza')	Push enviado al ciudadano

7	Tecnico → App Móvil	marcarEnAtencion(rep orte_id)	Estado → EN_ATENCION; timestamp registrado
8	Tecnico → App Móvil	adjuntarEvidenciaReso lucion(foto)	Imagen subida → URL almacenada
9	Tecnico → App Móvil	marcarResuelto(report e_id, comentario)	Estado → RESUELTO
10	API → ServicioNotificacion	enviarPush(ciudadano _id, 'Tu reporte fue resuelto – Ver foto')	Push con foto adjunta
11	API → ReporteService	calcularTiempoAtencio n(reporte_id)	Tiempo total: 4h 23min registrado
12	Sistema → Timer	programarSolicitudValo racion(ciudadano_id, reporte_id, delay=2h)	Tarea programada

4.4 Diagrama de Secuencia – Seguimiento y Cierre



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	Timer → ServicioNotificacion	enviarSolicitudValoracion(ciudadano_id, reporte_id)	Push: '¿Cómo fue la atención?'
2	Ciudadano → App Móvil	enviarValoracion(report e_id, puntuacion=4, comentario)	Valoración almacenada
3	API → KPIService	actualizarSatisfaccion(unidad_id, puntuacion)	KPI actualizado en dashboard

4a [puntuacion $\leq$ 2]	API → ServicioNotificacion	alertarSupervisor(reporte_id, puntuacion)	Alerta enviada al supervisor
4b [puntuacion > 2]	API → BaseDatos	UPDATE reporte SET estado='CERRADO'	Estado → CERRADO
5	Sistema → ReporteEstadistico	actualizarMetricasMensuales()	KPIs actualizados
6 [día 1 c/mes]	Scheduler → ReporteEstadistico	generarInformeTransparencia(periodo)	PDF generado y publicado en portal

Capítulo 5. Metodología de Trabajo – SCRUM

5.1 Definición y Justificación de la Metodología Scrum

Scrum es un marco de trabajo ágil definido por Ken Schwaber y Jeff Sutherland (Scrum Guide, 2020) que permite a equipos pequeños y multifuncionales entregar productos complejos de manera incremental a través de ciclos cortos de desarrollo llamados Sprints. Se fundamenta en tres pilares empíricos: Transparencia (todo el trabajo y su progreso es visible), Inspección (revisión continua de los artefactos) y Adaptación (ajuste rápido ante desviaciones).

Para el proyecto PIGIU, la adopción de Scrum se justifica técnicamente por las siguientes razones:

Criterio de Selección	Análisis para el Proyecto PIGIU
Requisitos cambiantes	La coordinación con la municipalidad y las normas de transparencia gubernamental pueden evolucionar durante el desarrollo. Scrum permite incorporar cambios entre sprints sin desestabilizar el proyecto.
Equipo pequeño y co-ubicado	Con 3 integrantes, Scrum elimina la burocracia de metodologías pesadas como RUP. La comunicación directa reduce el overhead de documentación formal.
Entrega de valor temprana	Organizando el backlog por procesos críticos (Registro → Derivación → Atención → Seguimiento), el Sprint 1 ya entrega un módulo funcional usable por ciudadanos.
Integración con herramienta Jira	Jira Software de Atlassian es la plataforma líder de gestión ágil, con soporte nativo para Scrum Boards, Backlogs priorizados, Sprints, Burndown Charts y reportes de velocidad, cubriendo el 100% de los artefactos Scrum requeridos.
Gestión de riesgo técnico	Las integraciones complejas (catastro GIS, PIGARS, FCM) se planifican en sprints específicos, permitiendo identificar impedimentos antes de que afecten el núcleo del sistema.
Validación con usuarios reales	Cada Sprint Review permite demostrar funcionalidades a representantes de la municipalidad y obtener feedback antes de invertir en el siguiente módulo.

5.2 Roles del Equipo Scrum

El equipo PIGIU opera como un Scrum Team autoorganizado con los siguientes roles y responsabilidades definidas:

Rol Scrum	Integrante	Responsabilidades Detalladas
Product Owner (PO)	Quispe Salas, Mónica	• Define, refina y prioriza el Product Backlog en Jira (campo 'Priority' y 'Story Points'). • Representa los intereses de la Municipalidad y los ciudadanos ante el equipo técnico. • Redacta y valida los criterios de aceptación de cada Historia de Usuario. • Aprueba o rechaza el incremento en cada Sprint Review. • Gestiona las relaciones con stakeholders externos (funcionarios municipales, MINAM). • Toma decisiones de negocio ante conflictos de prioridad entre funcionalidades.
Scrum Master (SM)	Lagones Briceño, Will, Salazar Yauri Javier	• Facilita las 5 ceremonias Scrum (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective, Backlog Refinement). • Administra el tablero Jira: crea el proyecto, configura workflows, estados de issues y permisos de usuario. • Elimina impedimentos del equipo: problemas técnicos, accesos, coordinaciones externas. • Monitorea el Burndown Chart en Jira para detectar desviaciones del Sprint Goal. • Protege al equipo del scope creep durante el sprint activo. • Facilita las Retrospectivas usando la técnica 'Start / Stop / Continue'.
Developer Team	Mandujano Coronel, Gean Carlos Lagones Briceño, Will Quispe Salas, Mónica, Salazar Yauri Javier	Gean Carlos (Backend Lead): Diseño e implementación de la API REST con Node.js, base de datos PostgreSQL+PostGIS, integraciones con servicios externos (FCM, S3, Catastro), lógica de negocio y pruebas unitarias con Jest. Will (Frontend/Móvil): Desarrollo de la app móvil en React Native y del panel administrativo en React.js; integración con la API REST; pruebas de usabilidad con usuarios reales. Mónica (QA/DevOps): Diseño y ejecución de pruebas funcionales y de carga; gestión de entornos con Docker; pipeline CI/CD; documentación técnica y de usuario final.

5.3 Gestión del Proyecto con Jira Software (Atlassian)

Jira Software es la herramienta oficial de gestión del proyecto PIGIU. Se utiliza el plan Jira Free (hasta 10 usuarios), suficiente para el equipo de 3 integrantes. A continuación se detalla la configuración completa del proyecto en Jira y cómo cada artefacto Scrum es gestionado dentro de la plataforma:

5.3.1 Configuración del Proyecto en Jira

Parámetro de Configuración	Valor Configurado en Jira
----------------------------	---------------------------

Nombre del Proyecto	PIGIU – Sistema de Reportes Urbanos Sostenibles
Clave del Proyecto (Key)	PIGIU
Tipo de Proyecto	Software – Scrum (Board con Backlog y Sprints)
URL del Proyecto	https://pigiui-team.atlassian.net/jira/software/projects/PIGIU/boards
Miembros del equipo	3 usuarios: gean.mandujano@, will.lagones@, monica.kispe@ (con roles PO, SM, Dev)
Idioma del tablero	Español
Estimación de issues	Story Points (Fibonacci: 1, 2, 3, 5, 8, 13, 21)
Velocidad de referencia	26 Story Points por Sprint de 2 semanas (basado en Sprint 1)

5.3.2 Tipos de Issues (Elementos de Trabajo) en Jira

El proyecto PIGIU utiliza la jerarquía nativa de Jira para organizar el trabajo en cuatro niveles:

Tipo de Issue	Icono Jira / Color	Uso en el Proyecto PIGIU
Epic	[Morado] Epic	Representa los 4 procesos principales del sistema: EPIC-01 Registro de Incidencias, EPIC-02 Derivación de Casos, EPIC-03 Atención de Casos, EPIC-04 Seguimiento Ciudadano. Cada Epic tiene su propia etiqueta de color en el tablero para identificación visual rápida.
Story (Historia de Usuario)	[Verde] Story	Una por cada Historia de Usuario (HU-01 a HU-18). Contiene: descripción en formato 'Como [actor], quiero [acción] para [beneficio]', criterios de aceptación en la sección 'Acceptance Criteria', estimación en Story Points y Epic de pertenencia. Es la unidad de trabajo principal del Sprint.
Task (Tarea)	[Azul] Task	Subtareas técnicas derivadas de cada Story. Ejemplos: 'Crear endpoint POST /reportes', 'Diseñar pantalla de captura en React Native', 'Escribir pruebas unitarias del servicio GPS'. Se asignan a un Developer específico con estimación en horas.
Bug	[Rojo] Bug	Defectos detectados durante las pruebas de QA. Contiene: pasos para reproducir, comportamiento esperado vs. actual, entorno de prueba, severidad (Crítico, Mayor, Menor, Trivial) y captura de pantalla adjunta. Se prioriza para corrección en el mismo sprint si es Crítico o Mayor.

5.3.3 Workflow (Flujo de Estados) de los Issues en Jira

Se configuró un workflow personalizado en Jira que refleja el ciclo de vida del desarrollo del equipo PIGIU:

Estado en Jira	Columna del Tablero	Definición y Criterio de Transición
TO DO	Backlog / Por Hacer	La Story o Task ha sido creada y priorizada en el Product Backlog, pero aún no ha sido seleccionada para un Sprint activo. Es el estado inicial de todos los issues.
SELECTED FOR DEV	Seleccionado	El issue fue incorporado al Sprint en el Sprint Planning. Tiene Developer asignado, estimación en SP y fecha de inicio planificada. Transición: Sprint Planning Meeting.
IN PROGRESS	En Desarrollo	El Developer ha iniciado activamente el trabajo. El Daily Scrum debe reportar el porcentaje de avance. La fecha de inicio real queda registrada automáticamente en Jira. Transición: el Developer mueve la tarjeta al iniciar el trabajo.
CODE REVIEW	Revisión de Código	El Developer ha completado la implementación y ha creado un Pull Request en GitHub. Otro miembro del equipo debe revisar el código antes de aprobar el merge. Jira se vincula al PR de GitHub mediante la integración nativa. Transición: al abrir el PR.
IN QA TESTING	En Pruebas	El código fue aprobado y desplegado en el entorno de staging. El responsable de QA (Mónica) ejecuta los casos de prueba definidos en los criterios de aceptación. Si encuentra defectos, crea un Bug issue vinculado a la Story. Transición: al hacer merge a develop.
DONE	Terminado	La Story cumple el Definition of Done (DoD): código mergeado a main, pruebas pasadas al 100%, criterios de aceptación validados por el PO, documentación actualizada, sin bugs Críticos ni Mayores abiertos. Transición: aprobación del PO en Sprint Review.

5.3.4 Definition of Done (DoD) – Definición de Terminado

El equipo acordó la siguiente Definition of Done aplicable a todas las Stories del proyecto. Una Story solo puede transicionar a estado DONE en Jira cuando cumple TODOS los siguientes criterios:

1. El código fuente está completo, comentado y sube el Pull Request a la rama develop en GitHub.
2. El Pull Request fue revisado y aprobado por al menos un integrante del equipo (Code Review en Jira).
3. Todos los criterios de aceptación de la Story han sido probados y aprobados por el QA (evidencia en la Story de Jira como adjunto o comentario).

4. La cobertura de pruebas unitarias del módulo nuevo no baja del 70% (reporte de Jest adjunto al issue).
5. El incremento fue demostrado en el entorno de staging y el Product Owner lo aprobó formalmente en la Sprint Review.
6. No existen bugs de tipo Crítico o Mayor vinculados a la Story sin resolver.
7. La documentación técnica del módulo (README, Swagger/OpenAPI para endpoints) está actualizada.
8. El issue en Jira está en estado DONE y el Story Point actualizado si hubo re-estimación durante el sprint.

5.3.5 Gestión del Backlog en Jira

El Product Backlog se gestiona directamente en la vista 'Backlog' de Jira. La estructura de épicas e historias es la siguiente:

Epic (Jira)	ID Story	Título de la Historia de Usuario	Story Points	Prioridad Jira
EPIC-01 Registro de Incidencias	PIGIU-01	Registro de usuario ciudadano (correo / Google)	5	Highest
EPIC-01	PIGIU-02	Captura fotográfica de incidencia con validación de calidad	8	Highest
EPIC-01	PIGIU-03	Geolocalización automática GPS con ajuste manual en mapa	8	Highest
EPIC-01	PIGIU-04	Clasificación de incidencia por categoría y subcategoría	5	High
EPIC-01	PIGIU-05	Generación de ticket único y confirmación al ciudadano	3	High
EPIC-02 Derivación de Casos	PIGIU-06	Derivación automática de incidencias a unidad responsable	13	Highest
EPIC-02	PIGIU-07	Mapa interactivo de incidencias para panel administrativo	13	Highest

EPIC-02	PIGIU-08	Reasignación manual de casos con bitácora de auditoría	8	Medium
EPIC-02	PIGIU-09	Escalamiento automático por vencimiento de SLA	8	High
EPIC-03 Atención de Casos	PIGIU-10	Bandeja de casos asignados con detalle para técnico	8	Highest
EPIC-03	PIGIU-11	Actualización de estado en campo con modo offline	13	Highest
EPIC-03	PIGIU-12	Evidencia fotográfica obligatoria al marcar Resuelto	5	High
EPIC-03	PIGIU-13	Generación de reportes estadísticos PDF/Excel	8	Medium
EPIC-04 Seguimiento Ciudadano	PIGIU-14	Notificaciones push por cambio de estado del reporte	5	Highest
EPIC-04	PIGIU-15	Historial completo del reporte con línea de tiempo	5	High
EPIC-04	PIGIU-16	Valoración del servicio (1-5 estrellas) post-resolución	3	Medium
EPIC-04	PIGIU-17	Dashboard ejecutivo con KPIs en tiempo real	8	Medium
EPIC-04	PIGIU-18	Publicación automática de informe de transparencia mensual	8	High

5.4 Planificación de Sprints en Jira

La planificación de sprints se realiza en la vista 'Sprint Planning' de Jira, donde el equipo selecciona los issues del Backlog y los incorpora al Sprint activo. Se definieron 7 sprints (incluyendo Sprint 0 de preparación):

SPRINT 0 – Configuración y Arquitectura Base

Atributo	Detalle
Duración	1 semana (Semana 1 del proyecto)
Sprint Goal	Tener el entorno de desarrollo completamente configurado y la arquitectura base del sistema definida y documentada, de modo que el equipo pueda comenzar a desarrollar funcionalidades en Sprint 1 sin impedimentos técnicos.
Issues en Jira (Tasks)	TASK-001: Crear repositorio GitHub con estructura de carpetas (monorepo) TASK-002: Configurar proyecto Jira con board, workflow y permisos TASK-003: Diseño y creación de la base de datos PostgreSQL+PostGIS TASK-004: Configurar entorno Docker (docker-compose.yml para dev/staging) TASK-005: Configurar pipeline CI/CD básico con GitHub Actions TASK-006: Configurar proyecto React Native y React.js (boilerplate) TASK-007: Definir estándares de código (ESLint, Prettier, convención de commits)
Capacidad del equipo	3 developers × 5 días × 8h = 120 horas totales disponibles
Criterio de éxito	Todos los entornos corren sin errores; la base de datos acepta conexiones; el pipeline CI pasa en verde; el tablero Jira está operativo.

SPRINT 1 – Registro de Incidencias (MVP Ciudadano)

Atributo	Detalle
Duración	2 semanas (Semanas 2-3)
Sprint Goal	El ciudadano puede registrarse, iniciar sesión y registrar una incidencia con foto y GPS, recibiendo su número de ticket de confirmación. Al finalizar el sprint, se tiene un flujo de reporte funcional de extremo a extremo.

Issues seleccionados	PIGIU-01 (5 SP) – Registro/Login ciudadano PIGIU-02 (8 SP) – Captura fotográfica PIGIU-03 (8 SP) – Geolocalización GPS PIGIU-04 (5 SP) – Categorización de incidencias PIGIU-05 (3 SP) – Generación de ticket
Total Story Points	29 SP
Capacidad del equipo	3 developers × 10 días × 6h productivas = 180 horas Velocidad objetivo: 29 SP
Distribución de tareas en Jira	Gean: Backend API (endpoints /auth, /reportes, /categorias, validación GPS) Will: App React Native (pantallas de registro, login, nuevo reporte, mapa) Mónica: QA (casos de prueba de todos los CAs definidos, pruebas de usabilidad con 5 personas externas)
Ceremonias Scrum en Jira	Sprint Planning: Día 1, 2h – selección de issues y asignación Daily Scrum: Cada día, 15 min – actualización del tablero Jira Sprint Review: Día 10, 1h – demo a stakeholder municipal Retrospectiva: Día 10, 45 min – 'Start/Stop/Continue'
Definition of Done aplicada	Los 5 stories en estado DONE; app funciona en Android e iOS; backend desplegado en staging; PO aprobó la demo.

SPRINT 2 – Derivación Automática y Panel Administrativo

Atributo	Detalle
Duración	2 semanas (Semanas 4-5)
Sprint Goal	Los funcionarios municipales pueden visualizar incidencias en el mapa interactivo, recibir casos derivados automáticamente y gestionar el ciclo de vida de los reportes desde el panel web.
Issues seleccionados	PIGIU-06 (13 SP) – Derivación automática PIGIU-07 (13 SP) – Mapa interactivo PIGIU-08 (8 SP) – Reasignación manual PIGIU-09 (8 SP) – Escalamiento por SLA
Total Story Points	42 SP
Nota de capacidad	Este sprint tiene alta carga (42 SP). Se activa la regla Scrum de re-estimación: si al día 7 el Burndown muestra riesgo, el PO puede negociar mover PIGIU-08 o PIGIU-09 al siguiente sprint manteniendo el Sprint Goal principal.

Distribución de tareas en Jira	Gean: Motor de derivación automática, WebSockets para mapa en tiempo real, lógica de escalamiento SLA, integración PostGIS para queries geoespaciales Will: Panel web React.js con Leaflet.js, componentes de mapa, filtros, popup de detalle, cambio de estado desde panel Mónica: Pruebas de carga del mapa (JMeter con 1000+ puntos), validación de derivación con distintas categorías
Riesgos identificados en Jira	RISK-001: La integración WebSockets puede requerir configuración adicional en el servidor; contingencia: polling cada 30s como fallback.

SPRINT 3 – App del Técnico y Modo Offline

Atributo	Detalle
Duración	2 semanas (Semanas 6-7)
Sprint Goal	El técnico operativo puede recibir, gestionar y resolver casos desde su dispositivo móvil, incluso sin conexión a Internet, adjuntando evidencia fotográfica geolocalizada al marcar un caso como resuelto.
Issues seleccionados	PIGIU-10 (8 SP) – Bandeja de casos para técnico PIGIU-11 (13 SP) – Actualización de estado offline PIGIU-12 (5 SP) – Evidencia fotográfica al resolver
Total Story Points	26 SP
Tecnología clave a implementar	SQLite local en React Native para almacenamiento offline; Redux-Persist para sincronización de estado; lógica de merge de conflictos al recuperar conexión.

SPRINT 4 – Seguimiento Ciudadano y Dashboard

Atributo	Detalle
Duración	2 semanas (Semanas 8-9)
Sprint Goal	Los ciudadanos reciben notificaciones en tiempo real sobre el estado de sus reportes y pueden valorar el servicio. La alcaldía dispone de un dashboard ejecutivo con KPIs actualizados automáticamente.

Issues seleccionados	PIGIU-14 (5 SP) – Notificaciones push PIGIU-15 (5 SP) – Historial de reporte PIGIU-16 (3 SP) – Valoración del servicio PIGIU-17 (8 SP) – Dashboard ejecutivo KPIs
Total Story Points	21 SP
Integración clave	Firestore Cloud Messaging (FCM) para notificaciones push; Recharts o Chart.js para el dashboard en React.js; sistema de colas para envíos masivos de notificaciones.

SPRINT 5 – Reportes, Transparencia e Integraciones Normativas

Atributo	Detalle
Duración	2 semanas (Semanas 10-11)
Sprint Goal	El sistema genera reportes estadísticos en PDF/Excel, publica el informe de transparencia mensual automáticamente y valida incidencias contra las capas GIS del catastro y las rutas PIGARS.
Issues seleccionados	PIGIU-13 (8 SP) – Reportes estadísticos PDF/Excel PIGIU-18 (8 SP) – Publicación informe transparencia RD-01 (integración normativa Ley 27806) RD-02 (integración rutas PIGARS) RD-03 (validación catastro GIS)
Total Story Points	24 SP + tareas técnicas de integración
Riesgo principal	La obtención de la API del catastro municipal requiere gestión institucional (carta de convenio). Contingencia: usar GeoJSON estático del IGN como sustituto temporal.

SPRINT 6 – Pruebas Finales, Hardening y Despliegue a Producción

Atributo	Detalle
Duración	1 semana (Semana 12)
Sprint Goal	El sistema PIGIU pasa todas las pruebas de aceptación del cliente, cumple los RNFs de rendimiento y seguridad definidos, y queda desplegado en el entorno de producción listo para el piloto en El Tambo.

Issues en Jira	TASK-FINAL-01: Prueba de carga con Apache JMeter (10,000 usuarios concurrentes) TASK-FINAL-02: Auditoría de seguridad con OWASP ZAP TASK-FINAL-03: Pruebas de usabilidad final con 10 usuarios ciudadanos TASK-FINAL-04: Corrección de bugs críticos encontrados en pruebas finales TASK-FINAL-05: Configuración del servidor de producción (AWS EC2 / Docker) TASK-FINAL-06: Generación de documentación final de usuario y técnica TASK-FINAL-07: Presentación del sistema al stakeholder municipal
Entregable final	Sistema PIGIU en producción + Manual de usuario (app ciudadana y panel) + Manual de administración + Informe de pruebas + Presentación ejecutiva

5.5 Herramientas Utilizadas en el Proyecto

El proyecto PIGIU emplea un ecosistema de herramientas especializadas, cada una seleccionada por su pertinencia técnica y su integración con el flujo de trabajo Scrum:

Herramienta	Categoría	Uso Específico en el Proyecto PIGIU
Jira Software (Atlassian)	Gestión Ágil [HERRAMIENTA PRINCIPAL]	Gestión completa del proyecto Scrum: Product Backlog con épicas e historias de usuario priorizadas, Sprint Planning, Scrum Board (Kanban para el sprint activo), Burndown Chart para monitoreo de avance, Velocity Chart para análisis de capacidad entre sprints, gestión de bugs, reportes de progreso. Integrado con GitHub para trazabilidad entre commits y issues. URL: https://pigiui-team.atlassian.net
Confluence (Atlassian)	Documentación	Wiki del proyecto: Actas de reuniones (Sprint Planning, Review, Retrospectiva), Especificación de arquitectura del sistema, Guías de instalación del entorno de desarrollo, Decisiones de diseño (ADR – Architecture Decision Records), Manual de usuario final. Integrado con Jira para vincular páginas a épicas o stories.
Draw.io (diagrams.net)	Modelado UML	Creación de todos los diagramas UML del proyecto: Diagrama de Contexto, Diagrama de Casos de Uso, Diagrama de Actividad con swimlanes, Diagrama de Clases completo, Diagramas de Secuencia. Los archivos .drawio se versionan en el repositorio GitHub y se incrustan en las páginas de Confluence. URL: https://app.diagrams.net
dbdiagram.io	Modelado de BD	Diseño visual del Modelo Entidad-Relación (E-R) de la base de datos PostgreSQL usando la sintaxis DBML (Database Markup Language). Permite generar automáticamente el script SQL de creación de tablas. El diagrama es exportado en PNG e incrustado en el

		informe técnico y en Confluence. URL: https://dbdiagram.io
GitHub	Control de versiones	Repositorio de código fuente con estrategia GitFlow: rama main (producción), develop (integración), feature/PIGIU-XX (por historia de usuario). Pull Requests vinculados a issues de Jira mediante Smart Commits (ej: 'git commit -m "PIGIU-02: validación resolución imagen"]'). GitHub Actions para CI/CD. URL: https://github.com/pigiu-team/pigiu-app
Figma	Diseño UI/UX	Diseño de los wireframes y prototipos de alta fidelidad de la app móvil y el panel web. Los diseños son validados con el Product Owner antes del inicio de cada sprint. Enlazados en Jira como adjuntos a las stories de frontend para referencia del Developer. URL: https://www.figma.com
Postman	Testing de API	Documentación y prueba de todos los endpoints REST de la API PIGIU. La colección de Postman incluye todos los endpoints con ejemplos de request/response, variables de entorno (dev, staging, prod) y tests automatizados que validan los criterios de aceptación de los RFs backend.
Apache JMeter	Pruebas de carga	Ejecución de pruebas de rendimiento para validar los RNFs: carga del mapa con 1,000+ incidencias en < 3 segundos, soporte de 10,000 usuarios concurrentes. Los reportes de JMeter (TPS, tiempo de respuesta, porcentaje de errores) se adjuntan como evidencia en Jira en las tasks de QA del Sprint 6.
Slack	Comunicación	Canal de comunicación asíncrona del equipo. Integrado con Jira (notificaciones de cambio de estado de issues) y GitHub (notificaciones de PR y CI). Canales: #general, #pigiu-backend, #pigiu-frontend, #pigiu-qa, #pigiu-daily.

5.6 Estructura Visual del Tablero Jira – Scrum Board PIGIU

El tablero Scrum de Jira es el centro de operaciones del equipo. A continuación se describe la estructura de columnas configurada en el proyecto PIGIU y un ejemplo de cómo se vería durante el Sprint 1 activo:

TO DO	IN PROGRESS	CODE REVIEW	IN QA TESTING	BLOCKED	DONE
PIGIU-01 Registro usuario 5 SP Highest Asign: Mónica	PIGIU-02 Captura foto 8 SP Highest Asign: Gean [2 días restantes]	PIGIU-03 Módulo GPS 8 SP Highest Asign: Will PR #12 en revisión	PIGIU-01 Tests: 12/12 ✓ Asign: Mónica Esperando PO	PIGIU-07 13 SP BLOQUEO: API catastro aun pendiente	PIGIU-05 Ticket generado 3 SP ✓ PO aprobado ✓ Tests 100% ✓

Cada tarjeta (card) del tablero muestra: ID del issue (PIGIU-XX), título resumido, Story Points, avatar del Developer asignado, prioridad con color (rojo=Highest, naranja=High, azul=Medium) y etiqueta de Epic de pertenencia en la parte superior de la tarjeta.

5.6.1 Integración Jira + GitHub con Smart Commits

El equipo configuró la integración nativa Jira-GitHub para trazabilidad total entre código e issues. Regla de equipo: ningún commit puede existir sin un issue de Jira asociado. Los Smart Commits permiten controlar el tablero directamente desde la terminal:

Acción del Developer	Sintaxis del Smart Commit en Git	Efecto Automático en Jira
Inicio de desarrollo de un issue	git commit -m "PIGIU-02 #in-progress: inicio módulo de captura fotográfica con validación"	El issue PIGIU-02 pasa automáticamente a columna IN PROGRESS. El commit aparece vinculado en la sección 'Development' del issue.
Apertura de Pull Request para revisión	Título del PR: "PIGIU-03: Módulo GPS con geolocalización automática [Closes PIGIU-03]"	El PR queda vinculado al issue PIGIU-03 en Jira. El SM y el revisor reciben notificación para Code Review.
PR aprobado, mover a QA	git commit -m "PIGIU-01 #in-qa-testing: merge feature/PIGIU-01 a develop post code review"	Issue pasa a IN QA TESTING. QA (Mónica) recibe alerta automática en Jira y Slack #pigiui-qa.
Issue completado y aprobado por PO	git commit -m "PIGIU-05 #done: criterios de aceptación verificados, PO aprobó en review"	Issue transiciona a DONE. Jira registra tiempo total de desarrollo automáticamente. Story Points suman al velocity chart.
Reporte de bug vinculado a un issue	Título del Bug en Jira: "BUG-003: La foto no valida resolución en dispositivos Android < 10 [Linked: PIGIU-02]"	Bug creado y vinculado al issue PIGIU-02. Si es severidad Crítica, el SM recibe alerta inmediata y el issue padre pasa a BLOCKED hasta resolución.

5.6.2 Burndown Chart y Control de Velocidad del Sprint

Jira genera automáticamente los gráficos de monitoreo del sprint que el Scrum Master revisa cada mañana antes del Daily Scrum:

Gráfico en Jira	Qué muestra	Cómo lo usa el equipo PIGIU
-----------------	-------------	-----------------------------

Burndown Chart (por Sprint)	Eje Y: Story Points restantes. Eje X: días del sprint. Línea gris = progreso ideal. Línea azul = progreso real del equipo.	Si al día 7 la línea azul está por encima de la gris, el SM activa alerta y el PO negocia remover el issue de menor prioridad del sprint para proteger el Sprint Goal.
Velocity Chart (histórico)	SP comprometidos vs. SP completados en cada sprint anterior, mostrado como barras comparativas.	El equipo usa el promedio de los 3 últimos sprints como referencia de capacidad para el siguiente Sprint Planning. Velocidad de referencia PIGIU: 26 SP/sprint.
Cumulative Flow Diagram (CFD)	Acumulación de issues por columna a lo largo del tiempo, visualizadas como áreas de color apiladas.	Si la banda 'IN PROGRESS' crece sin que la de 'DONE' también crezca, indica cuello de botella. El SM genera una Task de investigación en Jira para el siguiente Daily.
Sprint Report (post-sprint)	Resumen automático al cerrar el sprint: issues DONE, issues no completados, SP entregados, comentarios.	Se comparte el enlace del Sprint Report con el docente como evidencia de trabajo en Jira. Los issues no completados se mueven al siguiente sprint con re-priorización del PO.

5.6.3 Notificaciones y Alertas Automáticas Configuradas en Jira

Evento en Jira	Destinatario	Canal de Notificación
Nuevo issue creado en el backlog	Todo el equipo PIGIU	Correo electrónico del equipo
Issue asignado a un Developer	Developer asignado únicamente	Correo + notificación push en app Jira Mobile
Issue movido a IN QA TESTING	Responsable de QA (Mónica)	Correo + Slack canal #pigi-u-qa
Issue bloqueado (estado BLOCKED)	Scrum Master (Will)	Correo urgente + Slack @will mensaje directo
Issue sin actualizar > 24h en IN PROGRESS	SM + Developer asignado	Correo automático de alerta de inactividad
Sprint cerrado / Sprint Report generado	Todo el equipo + acceso vista al docente	Correo con enlace al reporte en Jira
Pull Request aprobado (GitHub → Jira)	Developer que abrió el PR	Notificación Jira + Slack #pigi-u-backend
Bug Crítico o Mayor abierto	Todo el equipo + SM directamente	Alerta roja en Jira + mensaje Slack urgente

5.7 Ceremonias Scrum – Protocolo del Equipo PIGIU

El equipo PIGIU sigue el protocolo establecido en la Scrum Guide (2020) para las 5 ceremonias del marco. Cada ceremonia tiene un formato definido, un facilitador y produce artefactos específicos gestionados en Jira y Confluence:

Ceremonia	Frecuencia	Duración Máx.	Facilitador	Entradas (Inputs)	Salidas (Outputs)
Sprint Planning	Inicio de cada Sprint	Máx. 4h (2 semanas)	Scrum Master (Will)	Product Backlog priorizado en Jira; velocidad sprint anterior; capacidad del equipo.	Sprint Goal definido; Sprint Backlog en Jira con issues asignados y estimados.
Daily Scrum	Diario (Lun-Vie)	15 minutos	Scrum Master	Board de Jira actualizado; 3 preguntas: ¿Qué hice ayer?, ¿Haré hoy?, ¿Impedimento?	Board actualizado; blockers registrados en Jira; plan del día acordado.
Sprint Review	Fin de Sprint	Máx. 2h	Product Owner (Mónica)	Incremento funcionando en staging; issues DONE según DoD; stakeholder municipal.	Demo aprobada; issues a DONE; feedback documentado en Confluence; Backlog actualizado.
Sprint Retrospectiva	Fin de Sprint	45 minutos	Scrum Master (Will)	Burndown Chart de Jira; incidentes del sprint; feedback del Sprint Review.	Máx. 3 mejoras en Confluence; al menos 1 acción como Task en el próximo sprint.
Backlog Refinement	Mitad de Sprint	1 hora	PO + SM	Backlog con issues sin estimar; nuevos requerimientos identificados.	Issues estimados; CAs refinados; épicas divididas si son > 13 SP.

5.8 Criterios de Aceptación Detallados – Formato Gherkin

Los criterios de aceptación de cada Historia de Usuario se documentan en el campo 'Acceptance Criteria' del issue de Jira, siguiendo el formato estándar BDD (Behavior-Driven Development) Given-When-Then (Gherkin). A continuación se presentan los criterios de las historias de mayor criticidad:

PIGIU-02: Captura fotográfica de incidencia

Escenario 1	Captura exitosa de fotografía GIVEN que el ciudadano está autenticado y en la pantalla 'Nuevo Reporte' WHEN presiona el botón de cámara THEN el sistema abre la cámara nativa del dispositivo en modo foto AND muestra botones de 'Capturar', 'Voltear cámara' y 'Cancelar'
Escenario 2	Rechazo de imagen de baja calidad GIVEN que el ciudadano capturó una imagen WHEN la resolución de la imagen es inferior a 640x480 píxeles THEN el sistema muestra el mensaje: 'La imagen no tiene suficiente calidad. Por favor, tome una nueva foto con mejor iluminación' AND no permite avanzar al siguiente paso hasta capturar una imagen válida
Escenario 3	Compresión automática de imagen válida GIVEN que el ciudadano capturó una imagen con resolución mínima de 640x480px WHEN confirma la imagen THEN el sistema la comprime automáticamente al formato WebP con calidad 85% AND el archivo resultante no supera 1MB AND la imagen es almacenada en el servidor en menos de 5 segundos AND se muestra una vista previa de la imagen comprimida al ciudadano

PIGIU-06: Derivación automática de incidencias

Escenario 1	Derivación automática exitosa por categoría GIVEN que se ha registrado un nuevo reporte con categoría 'Residuos Sólidos' WHEN el sistema procesa el reporte THEN lo deriva automáticamente a la Subgerencia de Limpieza Pública en menos de 5 minutos AND el responsable de la unidad recibe notificación push y correo electrónico AND el estado del reporte cambia de 'Pendiente' a 'Asignado' AND el ciudadano recibe notificación indicando 'Su caso fue asignado a Limpieza Pública'
Escenario 2	Reasignación manual con registro en bitácora GIVEN que un reporte fue derivado automáticamente a la Unidad A WHEN un supervisor reasigna manualmente el caso a la Unidad B THEN el sistema registra en bitácora: usuario que reasigna, fecha/hora exacta, unidad origen, unidad destino AND el campo 'Motivo de reasignación' es obligatorio (no puede quedar vacío) AND ambas unidades (A y B) reciben notificación del cambio
Escenario 3	Escalamiento automático por inactividad GIVEN que un reporte está en estado 'Pendiente' sin ser derivado WHEN transcurren 60 minutos desde el registro THEN el sistema envía alerta automática al supervisor de guardia con los datos completos del reporte AND registra el evento de escalamiento en el log del sistema AND cambia la prioridad del reporte a 'Urgente' en el panel administrativo

PIGIU-11: Actualización de estado en campo con modo offline

Escenario 1	Actualización offline sincronizada al recuperar conexión GIVEN que el técnico está en campo sin conexión a Internet WHEN actualiza el estado de un reporte a 'En Atención' desde la app móvil THEN la app almacena el cambio en SQLite local con timestamp exacto AND muestra indicador visual 'Pendiente de sincronización' AND cuando se recupera la conexión a Internet, sincroniza automáticamente en menos de 30 segundos AND el panel administrativo refleja el cambio sin intervención manual
--------------------	--

Escenario 2	Bloqueo al intentar marcar 'Resuelto' sin evidencia GIVEN que el técnico intenta marcar un reporte como 'Resuelto' WHEN no ha adjuntado ninguna fotografía de evidencia de resolución THEN el sistema bloquea la transición de estado AND muestra el mensaje: 'Debe adjuntar al menos una fotografía del trabajo terminado para poder marcar este reporte como Resuelto' AND el botón 'Confirmar Resolución' permanece deshabilitado hasta adjuntar la foto
Escenario 3	Advertencia por geolocalización de foto fuera de rango GIVEN que el técnico adjunta la fotografía de resolución WHEN las coordenadas GPS de la foto están a más de 100 metros del punto reportado de la incidencia THEN el sistema muestra advertencia: 'La foto fue tomada a más de 100m del punto de la incidencia. ¿Confirma que corresponde a la misma incidencia?' AND el técnico debe confirmar explícitamente antes de poder marcar como Resuelto AND la discrepancia queda registrada en el historial del reporte

PIGIU-17: Dashboard ejecutivo con KPIs en tiempo real

Escenario 1	Visualización de KPIs actualizados en tiempo real GIVEN que el funcionario con rol 'Gerencia' accede al Dashboard Ejecutivo WHEN se resuelve un nuevo reporte en el sistema THEN el KPI 'Tasa de Resolución' se actualiza automáticamente en el dashboard en menos de 5 segundos sin necesidad de recargar la página AND todos los indicadores (TPPR, TPR, satisfacción, ranking de sectores) reflejan los datos del día actual
Escenario 2	Exportación del dashboard en PDF GIVEN que el funcionario visualiza el dashboard con los KPIs del mes WHEN selecciona 'Exportar PDF' y confirma el período (mes/año) THEN el sistema genera el documento PDF con todos los gráficos y tablas de KPIs en menos de 10 segundos AND el PDF incluye: encabezado institucional de la municipalidad, fecha de generación, período analizado y firma digital del sistema

Conclusiones y Recomendaciones

Conclusiones

1. La Plataforma Integral de Gestión de Incidencias Urbanas (PIGIU) representa una solución tecnológica pertinente y técnicamente viable para resolver el déficit de comunicación entre la ciudadanía y la administración municipal. Al digitalizar el ciclo completo de gestión de incidencias —desde el registro hasta el cierre con valoración— se establece un canal trazable, transparente y eficiente que fortalece la gobernanza local.
2. El análisis de requerimientos reveló que la problemática no es solo tecnológica sino también procesal: la ausencia de flujos de trabajo estandarizados dentro de la municipalidad es tan crítica como la falta de una app ciudadana. El diseño del sistema atiende ambas dimensiones mediante la definición de roles, estados y SLAs que estructuran la operación interna.
3. La aplicación de la metodología Scrum permitió estructurar el desarrollo en seis sprints progresivos, cada uno entregando valor funcional incremental y reduciendo el riesgo técnico asociado a integraciones complejas (catastro, GIS, notificaciones push). El backlog priorizado garantiza que las funcionalidades de mayor impacto ciudadano se desarrollen primero.
4. La alineación con el ODS 11 (Ciudades y Comunidades Sostenibles) no es nominal: el sistema contribuye directamente a las Metas 11.3 y 11.6 al promover la participación ciudadana en la planificación urbana y mejorar la gestión de residuos mediante integración con el PIGARS. Esto posiciona al proyecto como una iniciativa de ciudad inteligente aplicable a contextos de ciudades intermedias peruanas.
5. El marco normativo analizado (Ley 27806 de Transparencia, Ley 29733 de Protección de Datos, Ley 27314 de Residuos Sólidos, Ley 27972 Orgánica de Municipalidades) demuestra que el sistema no solo es técnicamente correcto sino legalmente necesario. Las municipalidades tienen obligaciones de transparencia que este sistema les permite cumplir de manera automatizada y auditable.

Lecciones Aprendidas

- La especificación detallada de criterios de aceptación en formato Given-When-Then (Gherkin) fue determinante para eliminar ambigüedades en los requerimientos y facilitar las pruebas de validación. Requerimientos vagos generan código incorrecto.
- La integración con sistemas externos (catastro, PIGARS) debe planificarse como sprint independiente y con tiempo adicional de coordinación institucional, ya que los plazos de obtención de APIs gubernamentales superan los ciclos de desarrollo privados.
- El diseño del modelo E-R con PostGIS desde la primera iteración evitó costosas refactorizaciones de base de datos al incorporar los datos geoespaciales como ciudadanos de primera clase del modelo de datos.
- La separación de la lógica de notificaciones en un microservicio independiente (ServicioNotificacion) facilitó el testing unitario y permitirá escalar esta funcionalidad sin impactar el núcleo del sistema.

Recomendaciones

- **Piloto controlado:** Se recomienda implementar el sistema en un distrito piloto (El Tambo, por su alta densidad poblacional y diversidad de incidencias) durante 3 meses antes del despliegue metropolitano, para validar la arquitectura en condiciones reales y ajustar los SLAs según la capacidad operativa real de la municipalidad.
- **Capacitación diferenciada:** Es indispensable diseñar programas de capacitación diferenciados: (a) Técnicos operativos: uso de la app en campo, modo offline y carga de evidencias; (b) Funcionarios: uso del panel, interpretación de KPIs y gestión de casos; (c) Ciudadanía: campaña de difusión en medios locales y ferias barriales.
- **Interoperabilidad regional:** A mediano plazo, se recomienda integrar PIGIU con la Plataforma Digital del Estado Peruano (GOB.PE) y con el sistema de Ventanilla Única Municipal, para que los reportes puedan ser también accesibles mediante los canales digitales del gobierno central.
- **Inteligencia artificial:** En una versión 2.0, incorporar un módulo de clasificación automática de incidencias mediante visión por computadora (análisis de la fotografía) que sugiera la categoría al ciudadano antes de que él la seleccione, reduciendo errores de clasificación y acelerando la derivación.
- **Métricas de impacto ODS:** Implementar un módulo de seguimiento de contribución a los ODS que permita reportar anualmente a organismos internacionales (CEPAL, BID, PNUD) el número de incidencias resueltas, toneladas de residuos atendidas y horas de alumbrado restauradas como indicadores de impacto del proyecto.

Evidencias

Google Meet Session:

WILL EDGAR LAGONES BRICENO (Presentando y anotando)

Informe_PIGIU

Nota: El símbolo → indica flujo de entrada al sistema; ← indica flujo de salida. Las entidades externas actúan como proveedores y consumidores de información del sistema PIGIU.

3.2 Diagrama de Casos de Uso – Vista General

El diagrama de casos de uso del sistema PIGIU representa la interacción entre los principales actores involucrados en la gestión de incidencias urbanas: el Ciudadano Reportante, el Técnico Operativo, el Funcionario Municipal y]

WhatsApp Chat:

Analisis
Gean, +51 946 290 442, +51 930 370 108, Tú

~ Monicaa<3 +51 930 370 108
Grupo 1, ODS
11: Ciudades y comunidades sostenibles _
Sistema de reportes urbanos 2:26 p. m.

~ Javier J. +51 946 290 442
Tienes la planilla? 2:26 p. m.

G
Gean
Ahora les paso 2:30 p. m.

Gean
docs.google.com
https://docs.google.com/document/d/1JCWKi5poO50Tgt2t0tlnz_UZUGZa62oM9Eusp=sharing
docs.google.com
https://docs.google.com/document/d/1JCWKi5poO50Tgt2t0tlnz_UZUGZa62oM9Bi6H9OYyTA/edit?usp=sharing 2:30 p. m.

Ya avance algunas cosas también pueden estar avanzando o cambiar o poner otros soluciones las que puse 2:31 p. m.

Referencias Bibliográficas

Congreso de la República del Perú. (2002). Ley N.° 27806 – Ley de Transparencia y Acceso a la Información Pública. Diario Oficial El Peruano.

Congreso de la República del Perú. (2011). Ley N.° 29733 – Ley de Protección de Datos Personales. Diario Oficial El Peruano.

Congreso de la República del Perú. (2000). Ley N.° 27314 – Ley General de Residuos Sólidos (modificada por D. Leg. N.° 1278). Diario Oficial El Peruano.

Congreso de la República del Perú. (2003). Ley N.° 27972 – Ley Orgánica de Municipalidades. Diario Oficial El Peruano.

Ministerio del Ambiente del Perú (MINAM). (2017). Decreto Supremo N.° 014-2017-MINAM – Reglamento del Decreto Legislativo N.° 1278. Lima: MINAM.

Naciones Unidas. (2015). Agenda 2030 para el Desarrollo Sostenible – ODS 11: Ciudades y Comunidades Sostenibles. Nueva York: ONU.

Sommerville, I. (2016). Ingeniería del Software (10.^a ed.). Pearson Educación.

Pressman, R., & Maxim, B. (2015). Ingeniería del Software: Un enfoque práctico (8.^a ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum: La guía definitiva de Scrum – Las reglas del juego. Scrum.org. <https://scrumguides.org>

Beck, K., et al. (2001). Manifiesto for Agile Software Development. Agile Alliance. <https://agilemanifesto.org>

Open Web Application Security Project (OWASP). (2021). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>

PostGIS Development Group. (2023). PostGIS 3.3 Manual. <https://postgis.net/documentation/>

Firebase. (2024). Firebase Cloud Messaging – Documentación oficial. Google Developers. <https://firebase.google.com/docs/cloud-messaging>

INDECI. (2023). Índice de Satisfacción Ciudadana Municipal – Ciudades Intermedias del Perú. Lima: Instituto Nacional de Defensa Civil.

BID (Banco Interamericano de Desarrollo). (2022). Ciudades Inteligentes: Innovación para el desarrollo urbano sostenible en América Latina. Washington D.C.: BID.